

A **storytelling-based lesson plan** for teaching **Chapter 6: Class Relationships in Java – Association, Aggregation, and Composition**. The approach combines storytelling with real-life analogies and code snippets. The entire session fits within **1 hour**, balancing storytelling, explanations, and code demonstrations.

Title: "The Town of Objectville" – Understanding Class Relationships in Java

Objective:

By the end of this 1-hour lesson, students will be able to:

- Differentiate **association**, **aggregation**, and **composition**
 - Recognize real-world examples of each
 - Implement them in Java through short, clear examples
-

1-Hour Breakdown:

Time	Activity
0–10 mins	Introduction with story
10–25 mins	Association concept and Java example
25–40 mins	Aggregation concept and Java example
40–55 mins	Composition concept and Java example
55–60 mins	Recap and questions

The Story Begins: "The Town of Objectville"

Welcome to Objectville — a small, structured town where everything is defined by classes. In Objectville, everyone has a role and a relationship with others. Some people just know each other, some depend on others, and some cannot exist without their connection. Today, you'll meet three citizens: Anna the Artist, Greg the Gallery Owner, and Cathy the Car.

1. Association – Just Friends (10–25 mins)

Anna the Artist lives in Objectville. She **knows** Greg the Gallery Owner. They work independently, but they **communicate** often. This is called **association** — they're connected but independent.

✓ Real-world analogy:

- A **Teacher** and a **School** — A teacher can work at a school, but they exist independently.

✓ Java Example:

```
class Artist {
    String name;
    public Artist(String name) {
        this.name = name;
    }

    public void collaborateWith(Gallery g) {
        System.out.println(name + " is showcasing art at " + g.getName());
    }
}

class Gallery {
    String name;
    public Gallery(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

✓ Usage:

```
Artist anna = new Artist("Anna");
Gallery modernArt = new Gallery("Modern Art Gallery");
anna.collaborateWith(modernArt);
```

📌 **Key Point:** Both objects exist **independently**, but they can interact.

2. 🧺 Aggregation – The Collection Holder (25–40 mins)

Greg the Gallery Owner collects works from many artists. He **has** a list of artists. But if the gallery shuts down, the artists still exist. That's **aggregation** — a whole with independent parts.

✅ Real-world analogy:

- A **Library** and **Books** — Library contains books, but books don't die if the library is gone.

✅ Java Example:

```
import java.util.List;
```

```
class Gallery {
    String name;
    List<Artist> artists;

    public Gallery(String name, List<Artist> artists) {
        this.name = name;
        this.artists = artists;
    }

    public void showArtists() {
        System.out.println("Artists in " + name + ":");
        for (Artist a : artists) {
            System.out.println("- " + a.name);
        }
    }
}
```

✅ Usage:

```
Artist a1 = new Artist("Liam");
Artist a2 = new Artist("Mia");
```

```
Gallery g = new Gallery("Sunset Gallery", List.of(a1, a2));  
g.showArtists();
```

 **Key Point:** One object **has-a** list of others, but the parts can live on.

3. **Composition – Life and Death Together (40–55 mins)**

Meet **Cathy the Car**. She is made up of an **Engine** and **Wheels**. If Cathy is destroyed, her parts go too. This is **composition** — strong ownership.

 **Real-world analogy:**

- A **House** and **Rooms** — If the house is destroyed, so are the rooms.

 **Java Example:**

```
class Engine {  
    public void start() {  
        System.out.println("Engine starts.");  
    }  
}  
  
class Car {  
    private Engine engine;  
  
    public Car() {  
        this.engine = new Engine(); // Created inside  
    }  
  
    public void drive() {  
        engine.start();  
        System.out.println("Car is driving.");  
    }  
}
```

 **Usage:**

```
Car car = new Car();  
car.drive();
```

 **Key Point:** The part (engine) is created and owned exclusively by the whole (car). If the car dies, so does the engine.

Recap – The Relationship Ladder (55–60 mins)

Relationship	Description	Lifespan Dependency
Association	Knows about	Independent
Aggregation	Has-a	Whole can die, parts live
Composition	Owns-a	Strong dependency – parts die with whole

Bonus Tip (if time allows):

Use **UML diagrams** with arrows:

- Association: plain line
 - Aggregation: hollow diamond
 - Composition: solid diamond
-

Conclusion:

“Just like the residents of Objectville, classes in Java have different ways of knowing, owning, and depending on each other. Once you understand their relationships, your OOP designs become much more powerful and meaningful.”
